

Curso: Programación Avanzada

Guía: 07

Título: Diseño de APIs RESTful desde el Dominio

Duración estimada: 120 minutos

Docente: Christian Andrés Candela

DISEÑO DE APIs RESTful DESDE EL DOMINIO

OBJETIVO

Aprender a diseñar APIs RESTful que **emergen del modelo de dominio**, en lugar de diseñar contratos API en aislamiento. Al finalizar, los estudiantes serán capaces de:

1. Comprender los fundamentos de **APIs REST**, métodos **HTTP** y **JSON**
2. Entender el enfoque **Domain-First** vs. **Contract-First**
3. Mapear operaciones del dominio a métodos HTTP
4. Diseñar DTOs (Data Transfer Objects) basados en entidades de dominio
5. Crear especificaciones OpenAPI que reflejen el modelo de dominio
6. Separar claramente el modelo de dominio del modelo de API



CAMBIO FUNDAMENTAL: En las guías anteriores (04, 05 y 06), modelamos el dominio y creamos los casos de uso. Ahora diseñamos la API que **expone** ese dominio al exterior.

FUNDAMENTOS

Antes de diseñar nuestra API, necesitamos comprender los conceptos fundamentales sobre los que se construye.

¿Qué es una API?

API (Application Programming Interface) es un **contrato** que define cómo dos sistemas se comunican entre sí. En el contexto web, una API permite que aplicaciones externas (frontends, apps móviles, otros servicios) interactúen con nuestro sistema sin conocer su implementación interna.

Analogía: Piense en un mesero de restaurante. Usted (cliente) hace un pedido al mesero, el mesero lleva el pedido a la cocina (servidor), y la cocina prepara y devuelve la respuesta a través del mesero. Usted nunca entra a la cocina — solo interactúa a través de la API (el mesero).

¿Qué es REST?

REST (Representational State Transfer) es un estilo arquitectónico para diseñar APIs web. No es un framework ni una librería, sino una **convención** que define cómo organizar la comunicación. Sus principios fundamentales:

- 1. **Recursos:** Todo es un recurso identificado por una URL
- 2. **Verbos HTTP:** Las acciones se expresan con métodos HTTP estándar
- 3. **Sin estado:** Cada petición es independiente (el servidor no guarda "sesión")
- 4. **Representaciones:** Los datos se intercambian en formatos como JSON

Referencia: Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.

Recursos y URLs

En REST, un **recurso** es cualquier concepto que se pueda nombrar e identificar. Los recursos se acceden mediante URLs (también llamadas endpoints):

Recurso	URL (Endpoint)
Todas las solicitudes	/api/solicitudes
Una solicitud específica	/api/solicitudes/42
El historial de una solicitud	/api/solicitudes/42/historial
Todos los usuarios	/api/usuarios

Estructura de una URL REST:

https://miapp.com/api/solicitudes/42/historial
└─ servidor ─┘ └─ recurso ─┘ └─ id ┘ └─ sub-recurso ┘

Métodos HTTP

Los **verbos HTTP** indican qué acción realizar sobre un recurso. Son el equivalente a las operaciones CRUD:

Método	Acción	Ejemplo	Descripción
GET	Leer	GET /solicitudes	Obtiene una lista de solicitudes
POST	Crear	POST /solicitudes	Crea una nueva solicitud
PUT	Reemplazar	PUT /solicitudes/42/clasificar	Reemplaza o ejecuta una operación completa
PATCH	Actualización parcial	PATCH /solicitudes/42/estado	Modifica un aspecto específico
DELETE	Eliminar	DELETE /solicitudes/42	Elimina una solicitud

Regla clave: La URL identifica **qué** recurso. El método HTTP indica **qué hacer** con él.

Códigos de Estado HTTP

El servidor responde con un **código numérico** que indica el resultado de la operación:

Rango	Significado	Códigos Comunes
2xx	✅ Éxito	200 OK , 201 Created , 204 No Content
4xx	❌ Error del cliente	400 Bad Request , 403 Forbidden , 404 Not Found
5xx	💣 Error del servidor	500 Internal Server Error

En nuestro sistema, los más relevantes son:

- **201 Created** → Solicitud creada exitosamente (incluye header `Location` con la URL del nuevo recurso)
- **400 Bad Request** → Los datos enviados son inválidos (ej: descripción muy corta)

- **404 Not Found** → La solicitud con ese ID no existe

JSON: Formato de Intercambio

JSON (JavaScript Object Notation) es el formato estándar para enviar y recibir datos en APIs REST. Es texto plano, legible para humanos y fácil de procesar para máquinas:

```
{  
  "codigo": "SOL-2024-00123",  
  "descripcion": "Solicito homologación de Programación I",  
  "estado": "REGISTRADA",  
  "canalOrigen": "CORREO_ELECTRONICO",  
  "fechaCreacion": "2024-03-15T10:30:00"  
}
```

Anatomía de una Petición y Respuesta HTTP

Una comunicación REST completa tiene dos partes:

Petición (Request):

POST /api/solicitudes HTTP/1.1	← Método + URL
Host: localhost:8080	← Servidor destino
Content-Type: application/json	← Formato del cuerpo
<pre>{ "tipoSolicitudId": 1, "descripcion": "Solicito homologación de la asignatura...", "canalOrigen": "CORREO_ELECTRONICO" }</pre>	← Cuerpo (Body) en JSON

Respuesta (Response):

HTTP/1.1 201 Created

← Código de estado

Location: /api/solicitudes/507f1f77

← URL del recurso creado

```
{
```

← Cuerpo de respuesta

```
  "id": "507f1f77",
```

```
  "codigo": "SOL-2024-00123",
```

```
  "estado": "REGISTRADA"
```

```
}
```

CONCEPTOS ESPECÍFICOS DEL DISEÑO

Con los fundamentos de REST claros, ahora definimos los conceptos específicos que guían el **diseño** de nuestra API.

Domain-First Design

Enfoque donde el diseño de la API emerge del **modelo de dominio** ya construido, garantizando que la API refleje fielmente las operaciones y conceptos del negocio.

DTOs (Data Transfer Objects)

Objetos diseñados específicamente para transportar datos entre capas (dominio ↔ API). No contienen lógica de negocio.

OpenAPI Specification

Estándar para describir APIs RESTful de forma independiente del lenguaje. Permite documentación interactiva y generación de código.

Referencia: [OpenAPI Specification 3.1](#)

CONTEXTUALIZACIÓN TEÓRICA

Enfoque Domain-First vs. Contract-First

Contract-First (Enfoque Tradicional)

1. Diseñar OpenAPI spec
2. Generar interfaces/stubs
3. Implementar controllers
4. Crear servicios
5. Crear entidades

Problema: La API dicta la estructura del dominio

Domain-First (Nuestro Enfoque)

1. Modelar dominio (Guías 04-05) 
2. Identificar operaciones del dominio 
3. Mapear dominio → API (ESTA GUÍA)
4. Crear DTOs
5. Documentar en OpenAPI

Beneficio: El dominio dicta la estructura de la API

PARTE 1: MAPEO DOMINIO → API

Del Agregado Solicitud a Endpoints REST

Partir del agregado `Solicitud` que ya modelamos:

```
// DOMINIO (ya construido en Guía 05)
public class Solicitud {
    // Operaciones del dominio
    public static Solicitud crear(...);
    public void clasificar(...);
    public void asignarResponsable(...);
    public void cambiarEstado(...);
    public void cerrar(...);
}
```

Mapeo a HTTP:

Operación del Dominio	HTTP Method	Endpoint	Razón
Solicitud.crear()	POST	/solicitudes	Crea un nuevo recurso
consultar todas	GET	/solicitudes	Lista recursos
consultar por ID	GET	/solicitudes/{id}	Obtiene un recurso
clasificar()	PUT/PATCH	/solicitudes/{id}/clasificar	Operación de negocio específica
asignarResponsable()	PUT/PATCH	/solicitudes/{id}/asignar	Operación de negocio específica
cambiarEstado()	PUT/PATCH	/solicitudes/{id}/estado	Cambio de estado
cerrar()	PUT/PATCH	/solicitudes/{id}/cerrar	Operación final
obtenerHistorial()	GET	/solicitudes/{id}/historial	Sub-recurso del agregado

Principios de Mapeo

- Recursos = Agregados:** Un agregado → un recurso principal

2. **Verbos HTTP ≠ Operaciones del dominio:** Usar sub-recursos o acciones
3. **Operaciones complejas:** Crear endpoints específicos (ej: /clasificar)
4. **Solo la raíz:** No exponer `EventoHistorial` como recurso independiente

PARTE 2: DISEÑO DE DTOs

¿Por qué DTOs separados del Dominio?

Razones:

1. **Estabilidad de la API:** Cambios internos del dominio no afectan la API
2. **Seguridad:** No exponer atributos internos (ej: `id` de MongoDB)
3. **Versionado:** Mantener múltiples versiones de DTOs
4. **Optimización:** Diferentes vistas del mismo agregado

¿Por qué Records para DTOs?

Los DTOs son **portadores de datos inmutables** — no tienen comportamiento ni estado mutable. Esto los hace candidatos ideales para **Java Records**:

```
// ❌ Clase tradicional: mucho boilerplate
public class MiDTO {
    private String campo;
    public String getCampo() { return campo; }
    public void setCampo(String campo) { this.campo = campo; }
}

// ✅ Record: conciso, inmutable, con equals/hashCode/toString automáticos
public record MiDTO(String campo) {}
```

Ventajas de records para DTOs:

1. **Inmutabilidad:** Un DTO no debería modificarse después de crearse
2. **Concisión:** Elimina getters, setters, constructores y `equals` / `hashCode`
3. **Claridad:** La firma del record muestra toda su estructura de un vistazo
4. **Compatibilidad:** Jackson y Bean Validation soportan records completamente

DTOs para el Agregado Solicitud

1. Request DTO: Crear Solicitud

```
package com.uniquindio.triage.application.dto.request;

import jakarta.validation.constraints.*;

/**
 * DTO para crear una nueva solicitud
 * Mapea a: Solicitud.crear()
 *
 * ! NO incluir: estado, fechaCreacion, codigo
 * Estos los maneja el dominio automáticamente
 */
public record CrearSolicitudRequest(

    @NotNull(message = "El tipo de solicitud es obligatorio")
    Long tipoSolicitudId,

    @NotBlank(message = "La descripción es obligatoria")
    @Size(min = 20, max = 1000,
        message = "La descripción debe tener entre 20 y 1000 caracteres")
    String descripcion,

    @NotNull(message = "El canal de origen es obligatorio")
    CanalOrigen canalOrigen

) {}
```

2. Request DTO: Clasificar Solicitud

```
package com.uniquindio.triage.application.dto.request;

import jakarta.validation.constraints.*;

/**
 * DTO para clasificar una solicitud existente
 * Mapea a: Solicitud.clasificar()
 */
public record ClasificarSolicitudRequest(

    @NotNull(message = "El tipo de solicitud es obligatorio")
    Long tipoSolicitudId,

    @NotNull(message = "La prioridad es obligatoria")
    PrioridadDTO prioridad,

    @NotBlank(message = "La justificación es obligatoria")
    @Size(min = 10, max = 500)
    String justificacion

) {}

/**
 * DTO anidado para Prioridad (Value Object)
 */
public record PrioridadDTO(

    @NotNull
    NivelPrioridad nivel, // CRITICA, ALTA, MEDIA, BAJA

    @NotBlank
    String justificacion

) {}
```

3. Response DTO: Solicitud Completa

```
package com.uniquindio.triage.application.dto.response;

import java.time.LocalDateTime;

/**
 * DTO de respuesta con información completa de una solicitud
 * Usado en: GET /solicitudes/{id}
 */
public record SolicitudDetalleResponse(
    String id,
    String codigo, // SOL-2024-00123
    TipoSolicitudDTO tipo,
    String descripcion,
    CanalOrigen canalOrigen,
    EstadoSolicitud estado,
    PrioridadDTO prioridad,
    UsuarioResumenDTO solicitante,
    UsuarioResumenDTO responsable, // puede ser null
    LocalDateTime fechaCreacion,
    LocalDateTime fechaUltimaActualizacion,
    int totalEventosHistorial
) {}

/**
 * DTO resumido de usuario (evita exponer todo el objeto Usuario)
 */
public record UsuarioResumenDTO(
    String id,
    String nombreCompleto,
    String email,
    String rol
) {}

/**
 * DTO resumido de TipoSolicitud
 */
public record TipoSolicitudDTO(
    Long id,
    String codigo,
```

```
    String nombre
) {}
```

4. Response DTO: Lista de Solicitudes

```
/**
 * DTO simplificado para listar solicitudes
 * Usado en: GET /solicitudes (con paginación)
 */
public record SolicitudResumenResponse(
    String id,
    String codigo,
    String tipoNombre,          // Solo el nombre, no el objeto completo
    String descripcionBreve,    // Primeros 100 caracteres
    EstadoSolicitud estado,
    NivelPrioridad nivelPrioridad,
    String solicitanteNombre,
    String responsableNombre,  // puede ser null
    LocalDateTime fechaCreacion
) {}
```

5. Response DTO: Historial

```
/**
 * DTO para eventos del historial
 * Usado en: GET /solicitudes/{id}/historial
 */
public record EventoHistorialResponse(
    Long secuencia,
    LocalDateTime fechaHora,
    TipoAccion accion,
    String usuarioNombre,      // Nombre del usuario que realizó la acción
    String observacion,
    EstadoSolicitud estadoAnterior, // puede ser null
    EstadoSolicitud estadoNuevo    // puede ser null
) {}
```

Patrón de Nomenclatura

Tipo	Patrón	Ejemplo
Request (crear)	Crear{Entidad}Request	CrearSolicitudRequest
Request (acción)	{Accion}{Entidad}Request	ClasificarSolicitudRequest
Response (detalle)	{Entidad}DetalleResponse	SolicitudDetalleResponse
Response (lista)	{Entidad}ResumenResponse	SolicitudResumenResponse

PARTE 3: ESPECIFICACIÓN OPENAPI

Estructura del Archivo openapi.yaml

```
openapi: 3.1.0
info:
  title: API Sistema de Triage y Gestión de Solicitudes Académicas
  description: |
    API RESTful para la gestión estructurada de solicitudes académicas.

    **Dominio:** Gestión de Solicitudes Académicas
    **Agregado Principal:** Solicitud

  version: 1.0.0
  contact:
    name: Programa de Ingeniería de Sistemas
    email: sistemas@uniquindio.edu.co

servers:
  - url: http://localhost:8080
    description: Servidor de desarrollo local
  - url: https://api-triage.uniquindio.edu.co
    description: Servidor de producción

tags:
  - name: Solicitudes
    description: Gestión del agregado Solicitud
  - name: Tipos de Solicitud
    description: Catálogo de tipos de solicitud
  - name: Usuarios
    description: Gestión de usuarios del sistema

paths:
  # ===== SOLICITUDES =====

  /api/solicitudes:
    post:
      tags:
        - Solicitudes
      summary: Crear una nueva solicitud
      description: |
        Registra una nueva solicitud en estado REGISTRADA.
```

```
    Mapea a la operación del dominio: `Solicitud.crear()`
operationId: crearSolicitud
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CrearSolicitudRequest'
      example:
        tipoSolicitudId: 1
        descripcion: "Solicito homologación de la asignatura Programación I cursa
        canalOrigen: CORREO_ELECTRONICO
responses:
  '201':
    description: Solicitud creada exitosamente
    headers:
      Location:
        description: URI de la solicitud creada
        schema:
          type: string
          example: /api/solicitudes/507f1f77bcf86cd799439011
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/SolicitudDetalleResponse'
  '400':
    description: Datos de entrada inválidos
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  '401':
    description: No autenticado

get:
  tags:
    - Solicitudes
  summary: Listar solicitudes (con paginación y filtros)
  description: |
    Obtiene lista de solicitudes con soporte de paginación, ordenamiento y filtros.
  operationId: listarSolicitudes
  parameters:
    - name: estado
```

```

    in: query
    description: Filtrar por estado
    schema:
      $ref: '#/components/schemas/EstadoSolicitud'
  - name: prioridad
    in: query
    description: Filtrar por nivel de prioridad
    schema:
      $ref: '#/components/schemas/NivelPrioridad'
  - name: solicitanteId
    in: query
    description: Filtrar por solicitante
    schema:
      type: string
  - name: page
    in: query
    description: Número de página (0-indexed)
    schema:
      type: integer
      default: 0
  - name: size
    in: query
    description: Tamaño de página
    schema:
      type: integer
      default: 20
      maximum: 100
  - name: sort
    in: query
    description: Campo de ordenamiento
    schema:
      type: string
      example: fechaCreacion,desc
responses:
  '200':
    description: Lista de solicitudes
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/PaginaSolicitudes'

```

```

/api/solicitudes/{id}:
  get:

```



```
tags:
  - Solicitudes
summary: Obtener detalle de una solicitud
operationId: obtenerSolicitudPorId
parameters:
  - name: id
    in: path
    required: true
    schema:
      type: string
responses:
  '200':
    description: Detalle de la solicitud
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/SolicitudDetalleResponse'
  '404':
    description: Solicitud no encontrada
```

/api/solicitudes/{id}/clasificar:

```
put:
  tags:
    - Solicitudes
  summary: Clasificar una solicitud
  description: |
    Asigna tipo y prioridad a una solicitud.
    Mapea a: `Solicitud.clasificar()`

    **Precondiciones:**
    - La solicitud debe estar en estado REGISTRADA
    - El usuario debe tener rol COORDINADOR
  operationId: clasificarSolicitud
  parameters:
    - name: id
      in: path
      required: true
      schema:
        type: string
  requestBody:
    required: true
    content:
      application/json:
```

```
    schema:
      $ref: '#/components/schemas/ClasificarSolicitudRequest'
responses:
  '200':
    description: Solicitud clasificada exitosamente
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/SolicitudDetalleResponse'
  '400':
    description: Estado inválido para clasificar
  '403':
    description: Permisos insuficientes
  '404':
    description: Solicitud no encontrada
```

/api/solicitudes/{id}/asignar:

```
put:
  tags:
    - Solicitudes
  summary: Asignar responsable a una solicitud
  description: |
    Designa un coordinador/docente como responsable.
    Mapea a: `Solicitud.asignarResponsable()`
  operationId: asignarResponsable
  parameters:
    - name: id
      in: path
      required: true
      schema:
        type: string
  requestBody:
    required: true
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/AsignarResponsableRequest'
  responses:
    '200':
      description: Responsable asignado exitosamente
```

/api/solicitudes/{id}/estado:

```
patch:
```

```
tags:
  - Solicitudes
summary: Cambiar estado de una solicitud
description: |
  Realiza una transición de estado válida.
  Mapea a: `Solicitud.cambiarEstado()`
operationId: cambiarEstado
parameters:
  - name: id
    in: path
    required: true
    schema:
      type: string
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/CambiarEstadoRequest'
responses:
  '200':
    description: Estado cambiado exitosamente
  '400':
    description: Transición de estado inválida
```

/api/solicitudes/{id}/cerrar:

```
put:
  tags:
    - Solicitudes
summary: Cerrar una solicitud
description: |
  Marca la solicitud como cerrada (estado final).
  Mapea a: `Solicitud.cerrar()`
operationId: cerrarSolicitud
parameters:
  - name: id
    in: path
    required: true
    schema:
      type: string
requestBody:
  required: true
  content:
```

```
    application/json:
      schema:
        $ref: '#/components/schemas/CerrarSolicitudRequest'
  responses:
    '200':
      description: Solicitud cerrada exitosamente
```

/api/solicitudes/{id}/historial:

```
get:
  tags:
    - Solicitudes
  summary: Obtener historial de una solicitud
  description: |
    Retorna todos los eventos del historial cronológicamente.
    Accede al historial del agregado Solicitud.
  operationId: obtenerHistorial
  parameters:
    - name: id
      in: path
      required: true
      schema:
        type: string
  responses:
    '200':
      description: Historial de la solicitud
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/EventoHistorialResponse'
```

===== COMPONENTES =====

```
components:
  schemas:
    # Request DTOs
    CrearSolicitudRequest:
      type: object
      required:
        - tipoSolicitudId
        - descripcion
        - canalOrigen
```

```
properties:
  tipoSolicitudId:
    type: integer
    format: int64
  descripcion:
    type: string
    minLength: 20
    maxLength: 1000
  canalOrigen:
    $ref: '#/components/schemas/CanalOrigen'
```

ClasificarSolicitudRequest:

```
type: object
required:
  - tipoSolicitudId
  - prioridad
  - justificacion
properties:
  tipoSolicitudId:
    type: integer
    format: int64
  prioridad:
    $ref: '#/components/schemas/PrioridadDT0'
  justificacion:
    type: string
    minLength: 10
    maxLength: 500
```

AsignarResponsableRequest:

```
type: object
required:
  - responsableId
properties:
  responsableId:
    type: string
```

CambiarEstadoRequest:

```
type: object
required:
  - nuevoEstado
  - observacion
properties:
  nuevoEstado:
```

```
    $ref: '#/components/schemas/EstadoSolicitud'
  observacion:
    type: string
    maxLength: 500
```

CerrarSolicitudRequest:

```
  type: object
  required:
    - observacion
  properties:
    observacion:
      type: string
      maxLength: 500
```

Response DTOs

SolicitudDetalleResponse:

```
  type: object
  properties:
    id:
      type: string
    codigo:
      type: string
      example: SOL-2024-00123
    tipo:
      $ref: '#/components/schemas/TipoSolicitudDTO'
    descripcion:
      type: string
    canalOrigen:
      $ref: '#/components/schemas/CanalOrigen'
    estado:
      $ref: '#/components/schemas/EstadoSolicitud'
    prioridad:
      $ref: '#/components/schemas/PrioridadDTO'
    solicitante:
      $ref: '#/components/schemas/UsuarioResumenDTO'
    responsable:
      $ref: '#/components/schemas/UsuarioResumenDTO'
    fechaCreacion:
      type: string
      format: date-time
    fechaUltimaActualizacion:
      type: string
      format: date-time
```

```
totalEventosHistorial:  
  type: integer
```

SolicitudResumenResponse:

```
type: object  
properties:  
  id:  
    type: string  
  codigo:  
    type: string  
  tipoNombre:  
    type: string  
  descripcionBreve:  
    type: string  
  estado:  
    $ref: '#/components/schemas/EstadoSolicitud'  
  nivelPrioridad:  
    $ref: '#/components/schemas/NivelPrioridad'  
  solicitanteNombre:  
    type: string  
  responsableNombre:  
    type: string  
  fechaCreacion:  
    type: string  
    format: date-time
```

EventoHistorialResponse:

```
type: object  
properties:  
  secuencia:  
    type: integer  
    format: int64  
  fechaHora:  
    type: string  
    format: date-time  
  accion:  
    $ref: '#/components/schemas/TipoAccion'  
  usuarioNombre:  
    type: string  
  observacion:  
    type: string  
  estadoAnterior:  
    $ref: '#/components/schemas/EstadoSolicitud'
```

```
estadoNuevo:
  $ref: '#/components/schemas/EstadoSolicitud'
```

```
PaginaSolicitudes:
  type: object
  properties:
    contenido:
      type: array
      items:
        $ref: '#/components/schemas/SolicitudResumenResponse'
    paginaActual:
      type: integer
    totalPaginas:
      type: integer
    totalElementos:
      type: integer
      format: int64
    tamanoPagina:
      type: integer
```

Value Objects como DTOs

```
PrioridadDT0:
  type: object
  required:
    - nivel
    - justificacion
  properties:
    nivel:
      $ref: '#/components/schemas/NivelPrioridad'
    justificacion:
      type: string
```

```
TipoSolicitudDT0:
  type: object
  properties:
    id:
      type: integer
      format: int64
    codigo:
      type: string
    nombre:
      type: string
```


UsuarioResumenDTO:

type: object

properties:

id:

type: string

nombreCompleto:

type: string

email:

type: string

rol:

type: string

Enumeraciones

EstadoSolicitud:

type: string

enum:

- REGISTRADA
- CLASIFICADA
- ASIGNADA
- EN_ATENCION
- ATENDIDA
- CERRADA

NivelPrioridad:

type: string

enum:

- CRITICA
- ALTA
- MEDIA
- BAJA

CanalOrigen:

type: string

enum:

- CSU
- CORREO_ELECTRONICO
- SAC
- PRESENCIAL
- TELEFONICO
- WEB

TipoAccion:

type: string

```
enum:
  - CREACION
  - CLASIFICACION
  - ASIGNACION
  - CAMBIO_ESTADO
  - PRIORIZACION
  - CIERRE
```

```
# Error Response
```

```
ErrorResponse:
  type: object
  properties:
    timestamp:
      type: string
      format: date-time
    status:
      type: integer
    error:
      type: string
    mensaje:
      type: string
    path:
      type: string
```

```
securitySchemes:
  bearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
```

```
security:
  - bearerAuth: []
```

PARTE 4: INTEGRACIÓN CON SPRING BOOT

1. Configuración del Proyecto

Archivo `build.gradle` :

```

plugins {
    id 'java'
    id 'org.springframework.boot' version '4.0.0'
    id 'io.spring.dependency-management' version '1.1.4'
    id 'org.openapi.generator' version '7.0.1'
}

group = 'com.uniquindio'
version = '1.0.0'
sourceCompatibility = '25'

repositories {
    mavenCentral()
}

dependencies {
    // Spring Boot
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    runtimeOnly 'com.h2database:h2'
    implementation 'org.springframework.boot:spring-boot-starter-validation'

    // OpenAPI/Swagger
    implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.2.0'

    // Mappers
    implementation 'org.mapstruct:mapstruct:1.6.3'
    annotationProcessor 'org.mapstruct:mapstruct-processor:1.6.3'

    // Testing
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

```

2. Configurar Swagger UI

Archivo application.yml :

```
springdoc:
  api-docs:
    path: /api-docs
  swagger-ui:
    path: /swagger-ui.html
    enabled: true
    tags-sorter: alpha
    operations-sorter: method
```

3. Acceder a Swagger UI

Ejecutar la aplicación y navegar a:

```
http://localhost:8080/swagger-ui.html
```

PARTE 5: EJERCICIOS PRÁCTICOS

Ejercicio 1: Diseñar DTOs para TipoSolicitud

Crear los DTOs necesarios para el recurso TipoSolicitud :

1. CrearTipoSolicitudRequest
2. ActualizarTipoSolicitudRequest
3. TipoSolicitudResponse

Requisitos:

- Incluir validaciones apropiadas
- El DTO debe reflejar las propiedades de la entidad TipoSolicitud

Ejercicio 2: Mapear Operación del Dominio a API

Tienes la siguiente operación en el dominio:

```
public void cambiarPrioridad(Prioridad nueva, String coordinadorId, String justificacio
```

Preguntas:

1. ¿Qué método HTTP usarías? (PUT, PATCH, POST)
2. ¿Cuál sería el endpoint?
3. Diseña el Request DTO necesario

► 💡 Ver solución sugerida

Ejercicio 3: Completar Especificación OpenAPI

Agrega la definición OpenAPI para el endpoint `/api/solicitudes/{id}/prioridad`

PRECAUCIONES Y RECOMENDACIONES

1. **DTOs ≠ Entidades:** Nunca exponer entidades del dominio directamente en la API
2. **Validación:** Usar Bean Validation (`@NotNull` , `@Size` , etc.) en Request DTOs
3. **Versionado:** Considerar versionado de API desde el inicio (ej: `/api/v1/solicitudes`)
4. **Documentación:** Incluir descripciones claras en OpenAPI
5. **Consistencia:** Mantener nomenclatura consistente entre dominio y API

EVALUACIÓN O RESULTADO

Al finalizar esta guía, los estudiantes deben:

1. ☒ Comprender los fundamentos de APIs REST, métodos HTTP y JSON
2. ☒ Comprender el enfoque domain-first
3. ☒ Haber mapeado las operaciones del dominio a endpoints REST
4. ☒ Haber creado DTOs separados del modelo de dominio
5. ☒ Tener una especificación OpenAPI completa del recurso Solicitud
6. ☒ Configurar Swagger UI en Spring Boot

PRÓXIMA ACTIVIDAD

En la **Guía 08**, implementaremos los **Controllers REST** que utilizan estos DTOs y conectan con el dominio a través de la capa de aplicación.

Preparación:

1. Repasar patrón **Mapper** (MapStruct)
2. Revisar anotaciones de Spring MVC (`@RestController` , `@RequestMapping` , etc.)

REFERENCIAS BIBLIOGRÁFICAS

1. OpenAPI Initiative. (2023). *OpenAPI Specification 3.1.0*. <https://spec.openapi.org/oas/v3.1.0>
2. Spring Framework. (2023). *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
3. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
4. Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
5. Springdoc. (2023). *Springdoc OpenAPI Documentation*. <https://springdoc.org/>